# Informe ejecutivo final

***Sistema de Gestión de Producción y Venta de Pollos de Engorde***

***Bogotá D.C., noviembre de 2026***

***Versión 1.0***

Gabriela Poveda, Dylan Mancipe y Juan Diego López
Universidad El Bosque · Bases de Datos 2
Docente: Ing., Esp., M.Ed. Christian Felipe Duarte

# 1. Problema

Las granjas avícolas pequeñas y medianas de Colombia suelen llevar los lotes, el alimento, los medicamentos, las ventas y los pagos en cuadernos u hojas de cálculo separadas. Sin un registro integrado no conocen el costo real ni la rentabilidad de cada lote, y deciden tarde sobre la compra de alimento, la fecha de venta y el cobro de la cartera.

# 2. Solución

Se construyó una aplicación web para una finca con una sola sede y varios galpones, que cubre el ciclo completo desde el ingreso de un lote hasta el cobro de su venta. Sus módulos son: galpones y lotes con mortalidad diaria y alerta cuando supera el 5 %; alimentación por etapas con inventario y costo promedio ponderado; sanidad; personal; gastos y costo del beneficio; clientes, pedidos repartidos entre lotes, ventas, pagos parciales, cartera vencida y entregas; un portal donde el cliente pide y paga en línea (Wompi en ambiente de pruebas), además del pago en efectivo que registra el personal; seguimiento diario del crecimiento en MongoDB; reportes de costo y rentabilidad por lote; usuarios, roles y permisos; y auditoría de todos los cambios. Quedan fuera la facturación electrónica, la nómina, el inventario de medicamentos, los sensores, la app móvil y los pagos con dinero real.

# 3. Arquitectura

La solución sigue una arquitectura de tres capas con una API REST [7]. La lógica del negocio vive en PostgreSQL; el backend la invoca y la expone; el frontend la presenta.

| Capa | Tecnología | Ubicación y función |
|---|---|---|
| Presentación | Angular 20, TypeScript 5.9 | Interfaz en español con 21 pantallas más el menú y gráficos del crecimiento. nginx la sirve en el equipo anfitrión. |
| API | Java 21, Spring Boot 4.1.1 | 19 controladores REST con JWT y permisos por operación; Swagger; correo con Thymeleaf. Corre como servicio systemd. |
| Datos relacionales | PostgreSQL 18 | VM 1 con Ubuntu 26.04 LTS Desktop (instalación mínima). Tablas, procedimientos, funciones, triggers, vistas y auditoría. |
| Datos documentales | MongoDB 7.0 | VM 2 con Ubuntu Server 26.04 LTS. Seguimiento diario del crecimiento. |
| Pagos | Wompi (sandbox) | Checkout externo y aviso firmado (webhook) que confirma el pago. |
| Publicación | nginx + Tailscale Funnel | nginx pasa /api al backend; Funnel publica el sistema por HTTPS. |

*Tabla 1. Componentes de la arquitectura.*

El backend se conecta a PostgreSQL con el rol pollos_app, que solo puede leer, escribir y ejecutar procedimientos; los objetos son de pollos_owner, un rol sin acceso. Las bases solo aceptan conexiones del equipo del backend (ufw). Ningún dato se guarda en las dos bases: MongoDB solo acepta un seguimiento si el lote existe y está activo en PostgreSQL.

# 4. Decisiones técnicas

| Decisión | Alternativas | Justificación |
|---|---|---|
| PostgreSQL 18 como RDBMS | MySQL, MariaDB, SQL Server | Ganó la matriz ponderada con 4,85 de 5 (MySQL 3,50; MariaDB y SQL Server 3,65) y se mantuvo primero en el análisis de sensibilidad [9]. Ofrece COMMIT/ROLLBACK en procedimientos, DDL transaccional, índices parciales y PL/pgSQL, cercano a PL/SQL [10]. |
| Lógica de negocio en la base de datos | Reglas en el backend | Las reglas se cumplen sin importar quién escriba los datos y cada operación es atómica [3]. El backend solo llama CALL sp_… y consulta vistas; Hibernate valida el esquema pero no lo crea. |
| Spring Boot en lugar de Node.js/Express | Node.js/Express (propuesta inicial) | Se cambió en el acta por la madurez del controlador JDBC para invocar procedimientos y por la seguridad integrada (Spring Security, validación, Swagger). |
| MongoDB solo para el seguimiento del crecimiento | Guardar todo en PostgreSQL o más módulos en MongoDB | El seguimiento cambia de estructura cada día (muestreos y novedades opcionales), solo se agrega y se analiza en conjunto [4], [5], [6]. Lo transaccional queda en PostgreSQL para no duplicar datos. |
| Ubuntu Server 26.04 LTS (VM 1 con Desktop mínima) | Debian 13, Rocky/AlmaLinux 10, FreeBSD | Ganó su matriz ponderada (4,65 de 5) por soporte LTS, repositorios oficiales y documentación. |
| MongoDB 7.0 en vez de 8.0 | Cambiar de SO o de kernel | MongoDB 8.0 no arranca con el kernel 7.0 de Ubuntu 26.04 (SERVER-121912 [11]). La 7.0 cubre todo lo que usa el proyecto. |
| Auditoría por triggers | Registro en el backend | Un trigger en cada tabla guarda quién, qué y cuándo, con los datos antes y después; la tabla es inmutable y captura también cambios hechos fuera de la aplicación. |
| Metodología híbrida | Cascada, Scrum o Kanban puros | Fases y hitos fijos del curso, con construcción en ocho incrementos verticales revisados por PR [8]. |

*Tabla 2. Principales decisiones técnicas.*

# 5. Resultados

- **Base relacional:** 34 tablas de negocio, catálogos y seguridad, normalizadas hasta 3FN [1], [2], con relaciones 1:1 (venta–entrega), 1:N (galpón–lote, cliente–pedido, venta–pago) y N:M (pedido–lote, jornada de beneficio–empleado, usuario–rol). Incluye 20 procedimientos, 48 funciones, 25 triggers declarados más el trigger de auditoría que se crea en cada tabla, y 22 vistas.
- **Pruebas:** 183 casos de prueba SQL con casos válidos e inválidos y 205 pruebas JUnit en el backend, todos aprobados.
- **MongoDB:** CRUD completo del seguimiento, índice único por lote y fecha, y agregación de la curva de crecimiento semanal ($match, $group por semana con $avg del peso).
- **Aplicación:** 21 pantallas en Angular más el menú y 19 controladores REST; reportes de costo con cinco componentes, rentabilidad por lote, ventas por cliente, cartera vencida, inventario de alimento y ocupación de galpones; pago en línea de punta a punta en sandbox.
- **Gestión:** desarrollo en ocho incrementos entre el 01/11 y el 09/11/2026, según el cronograma, con 13 PR en el backend y 9 en el frontend; los PR de los incrementos (#1 a #8 del backend y #2 a #8 del frontend) fueron aprobados por otro integrante. Javadoc publicado en GitHub Pages el 10/11/2026.

# 6. Lecciones aprendidas

- **Verificar la compatibilidad de versiones antes de instalar:** MongoDB 8.0 no arrancó en Ubuntu 26.04 por su kernel; revisar la matriz de plataformas soportadas habría evitado la reinstalación.
- **Las VM dependen del anfitrión:** Al suspender el computador, la VM quedaba con el reloj desfasado y carga alta; reiniciarla lo resolvía. Con poca RAM hubo que limitar la caché de MongoDB.
- **Unificar el entorno de desarrollo:** Las diferencias de finales de línea entre Windows y Linux (CRLF) y la configuración de CORS al publicar causaron errores que no aparecían en desarrollo.
- **Separar los cambios de reglas en scripts nuevos:** Cuando se permitieron varios lotes por galpón, el cambio se aplicó en un módulo nuevo y atómico, sin reescribir los anteriores.

# 7. Conclusiones

- El sistema cumple los requisitos técnicos del enunciado: base relacional en 3FN con procedimientos, funciones, triggers, transacciones y excepciones; MongoDB con CRUD y agregaciones justificado por la estructura variable del seguimiento; despliegue en máquinas virtuales Linux; y una aplicación web conectada a ambas bases.
- Llevar la lógica a PostgreSQL dio integridad y atomicidad garantizadas por el motor, y permitió probar cada regla con casos válidos e inválidos independientes de la aplicación.

- La finca puede conocer el costo real y la rentabilidad de cada lote, que era el problema de partida.
- La metodología híbrida y el flujo de Git con revisión por pares permitieron cumplir la entrega del 11/11/2026 y con evidencia de la participación de los tres integrantes.

# Referencias

[1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377-387, 1970, doi: 10.1145/362384.362685.

[2] R. Elmasri y S. B. Navathe, *Fundamentals of Database Systems*, 7.ª ed. Hoboken, NJ, EE. UU.: Pearson, 2016.

[3] T. Haerder y A. Reuter, "Principles of transaction-oriented database recovery," *ACM Computing Surveys*, vol. 15, no. 4, pp. 287-317, 1983.

[4] R. Cattell, "Scalable SQL and NoSQL data stores," *ACM SIGMOD Record*, vol. 39, no. 4, pp. 12-27, 2010.

[5] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, EE. UU.: O'Reilly Media, 2017.

[6] S. Bradshaw, E. Brazil y K. Chodorow, *MongoDB: The Definitive Guide*, 3.ª ed. Sebastopol, CA, EE. UU.: O'Reilly Media, 2019.

[7] R. T. Fielding, "Architectural styles and the design of network-based software architectures," tesis doctoral, Univ. of California, Irvine, CA, EE. UU., 2000.

[8] B. Boehm y R. Turner, *Balancing Agility and Discipline: A Guide for the Perplexed*. Boston, MA, EE. UU.: Addison-Wesley, 2003.

[9] T. L. Saaty, "How to make a decision: The analytic hierarchy process," *European Journal of Operational Research*, vol. 48, no. 1, pp. 9-26, 1990, doi: 10.1016/0377-2217(90)90057-I.

[10] PostgreSQL Global Development Group, *PostgreSQL 18 Documentation*, 2026. [En línea]. Disponible en: https://www.postgresql.org/docs/18/. [Consultado: 10-nov-2026].

[11] MongoDB Inc., "SERVER-121912," *MongoDB Jira*, 2026. [En línea]. Disponible en: https://jira.mongodb.org/browse/SERVER-121912. [Consultado: 11-nov-2026].